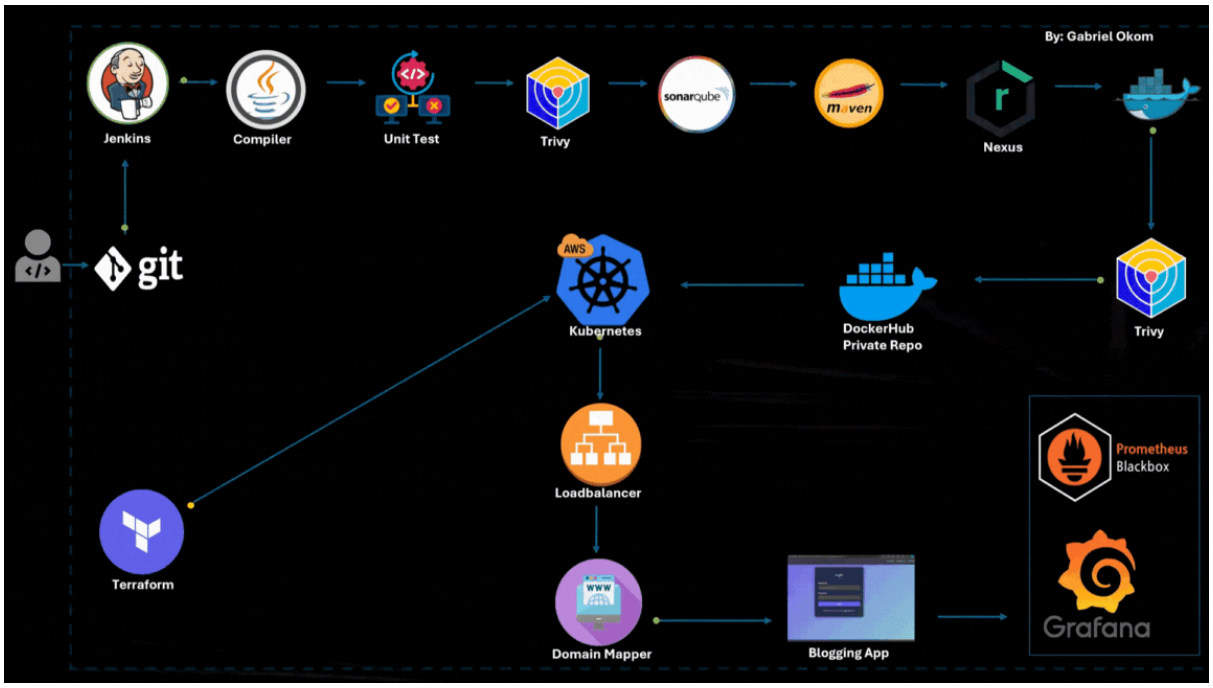
CICD PROJECT: Production Level Blog APP Deployment using EKS, Nexus, SonarQube, Trivy with Monitoring Tools



=> **Tools Used:**

- **Jenkins:** For managing the CI/CD pipeline.
- **SonarQube:** For static code analysis.
- **Nexus:** For managing dependencies and artifacts.
- **Trivy:** For scanning vulnerabilities in files and Docker images.
- **Docker:** To containerize applications.
- **Prometheus:** For monitoring metrics from services.
- **Blackbox Exporter:** For probing application availability.
- **Grafana:** For visualizing metrics.
- **Kubernetes (AWS EKS):** For managing containerized workloads.
- **Terraform:** For EKS deployment.

Prerequisites:

- Basic Understanding of CI/CD: Familiarity with Continuous Integration and Continuous Deployment.
- AWS Account: Access to create and manage EC2 instances and EKS.
- Git Knowledge: Experience using Git and GitHub.
- Linux Commands: Basic experience with terminal commands and SSH access.
- Jenkins, Docker, and Kubernetes knowledge: Understanding of basic setup and usage.

Table of Contents:

- **Step 1:** Set up Git Repository and create Security Token
- **Step 2:** Setup required servers (Jenkins, Sonarqube, Nexus, Monitoring tools)
- **Step 3:** Set up Jenkins, Sonarqube and Nexus
- **Step 4:** Install Jenkins Plugins, and Configure Nexus, Trivy, SonarQube and DockerHub to use Jenkins
- **Step 5:** Create a complete CICD pipeline
- **Step 6:** Create the EKS cluster, Install AWS CLI, Kubectl and Terraform
- **Step 7:** Assign a custom domain to the deployed application
- **Step 8:** Monitor the application

Step 1. Set up Git Repository and create Security Token

a.> **Create the Repo:** We will need to setup a private git repo, it is assumed you already know how to do one. If not click here to read the official GitHub docs. You can decide to make it *public or private* for production use, it is best to set it to private this way it is more secured and not exposed to the public.

However; by choice I will be leaving my repo public , this way you can get access to it and the source files used for this project. Repo Link

b.> **Create a Security Token:** After setting up the git repo, we will have to create a security token ; this will help us authenticate easily. Another major importance is that it ensures secured, managed access to your repositories without exposing your actual password.

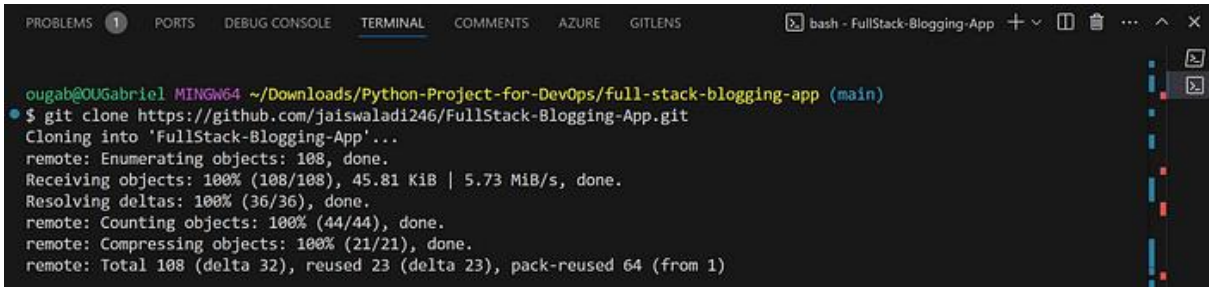
c.> **Install GitBash in your local system and clone the repo:**

Again, it is assumed you already know how to install Git Bash, it is quite easy to do this for Windows and Mac. Download the OS and follow the installation prompts. Click Here to get started and install Git. Having Git on your local system is a advisable because it makes it easier to push and commits code.

After installing Git Bash, we need to clone the repo we will be using, this repo contains the source code needed for this project. Click Here to CLONE it.

git clone https://github.com/ougabriel/full-stack-blogging-app.git

I am running the git clone command in VS Studio



When this is done make sure to cd into the project directory

cd FullStack-Blogging-App/

Step 2: Setup required servers (Jenkins, Sonarqube, Nexus, Monitoring tools)

Here, we are going to deploy 2 EC2 instance for Nexus and Sonarqube

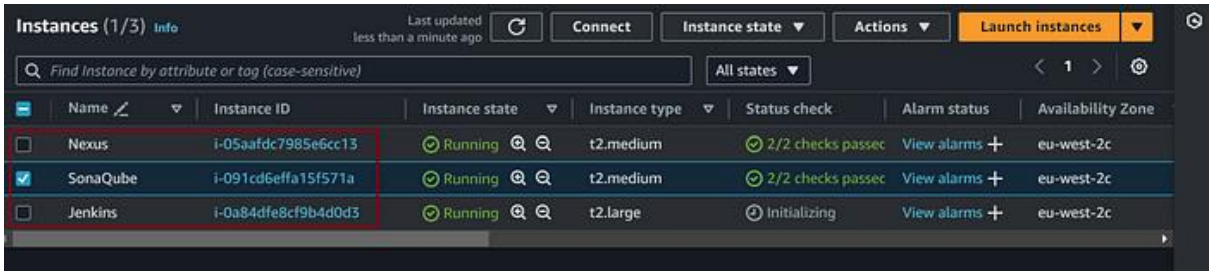
To create two t2.mediumUbuntu EC2 instances on AWS, follow these steps:

a.> Log in to AWS Console: Go to the AWS Management Console and sign in to your account.
b.> Navigate to EC2: In the search bar, type "EC2" and select EC2to go to the EC2 dashboard.
c.> Launch Instance: Click on Launch instances.
d. > Configure Instance Details: -Name: Give a name to your instances. -AMI: Choose an Amazon Machine Image (AMI) by selecting Ubuntu Server 20.04 LTS. -Instance type: Select t2.mediumfrom the dropdown.
e.> Key Pair (SSH login): -Select an existing key pair or create a new one to securely connect via SSH.
f.> Network settings: Create or Choose your preferred VPC and subnet. -Ensure that Auto-assign publi IP is enabled for external access. -Configure Security Group: Allow SSH (port 22) access by specifying your IP range of 2000 –11000.
g.> 7. Configure storage: -The storage size should be 20GB
h.> 8. Set the Number of Instances: -In the Number of instances field, enter 2 to create two instances.
i.> Launch Instances: -Review your settings and click Launch.
j.>10. Connect: -Once the instances are running, you can use the SSH keyto connect via terminal:

ssh -i /path/to/your-key.pem ubuntu@<public-ip>

Next; we are going to create a separate EC2 instance with a large storage size of 25GB and Instance type of t2.largefor Jenkins.

Now, repeate the same process and create an EC2 instance of size 25GB and Instance type of t2.large, use the same security group and dont forget to make sure Auto-assign publi IP is enabled for external access.



Best way to connect to the 3 instances is to use MobaXterm, a third party app that can be used to ssh into any system. Find the guide here.

Step 3: Set up Jenkins, Sonarqube and Nexus

3.1 Jenkins set up: To configure Jenkins for use, we need to install some few things.

a.> SSH into the Jenkins instance

ssh -i <path-to-your-keyp.pem ubuntu@<jenkins-vm-public-ip>

b.> Update and install java

sudo apt update

```
sudo apt install openjdk-17-jre-headless  -y
```

c.> Install Jenkins: Using scripts make it easier to install packages, and helps to save time spent on running single commands.

```
#!/bin/bash

# Update system packages
sudo apt-get update -y

# Install Java (Jenkins requires Java to run)
sudo apt-get install -y openjdk-11-jdk

# Import Jenkins GPG key and add Jenkins apt repository
curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io.key | sudo tee \
  /usr/share/keyrings/jenkins-keyring.asc > /dev/null

echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
  https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
  /etc/apt/sources.list.d/jenkins.list > /dev/null

# Update package lists to include Jenkins repository
sudo apt-get update -y

# Install Jenkins
sudo apt-get install -y jenkins

# Start Jenkins
sudo systemctl start jenkins

# Enable Jenkins to start at boot
sudo systemctl enable jenkins
```

Print the initial Jenkins admin password

echo "Jenkins installed successfully!"

echo "You can access Jenkins at http://<your-server-ip>:8080"

echo "Use the following command to retrieve your initial Jenkins admin password:"

echo "sudo cat /var/lib/jenkins/secrets/initialAdminPassword"

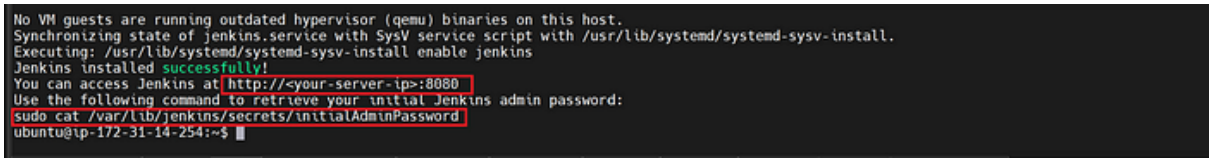
Run the following command to run the script

vi install_jenkins.sh #paste the script into the editor. Press Ctrl wq! to save and exit

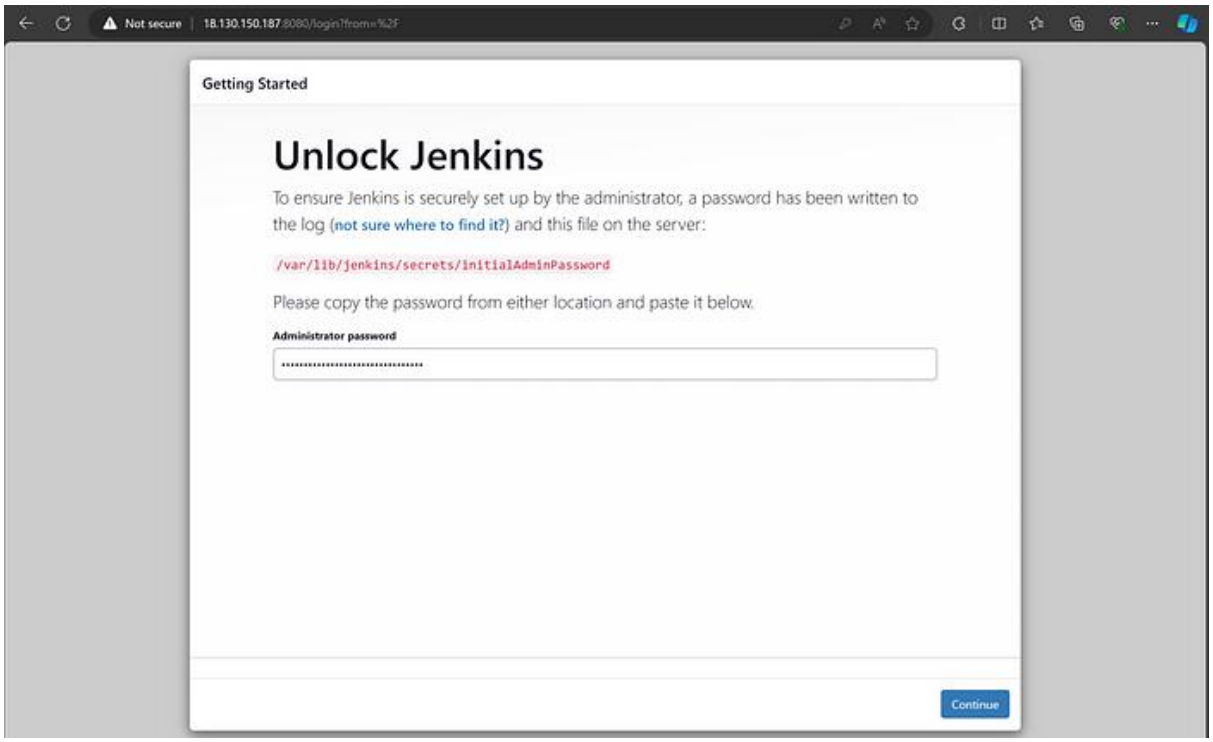
chmod +x install_jenkins.sh

./install_jenkins.sh

After the installation; you can login to Jenkins by using http://<your-jenkins-public-ip>:8080 . You can get the jenkins initial password here sudo cat /var/lib/jenkins/secrets/InitialAdminPassword

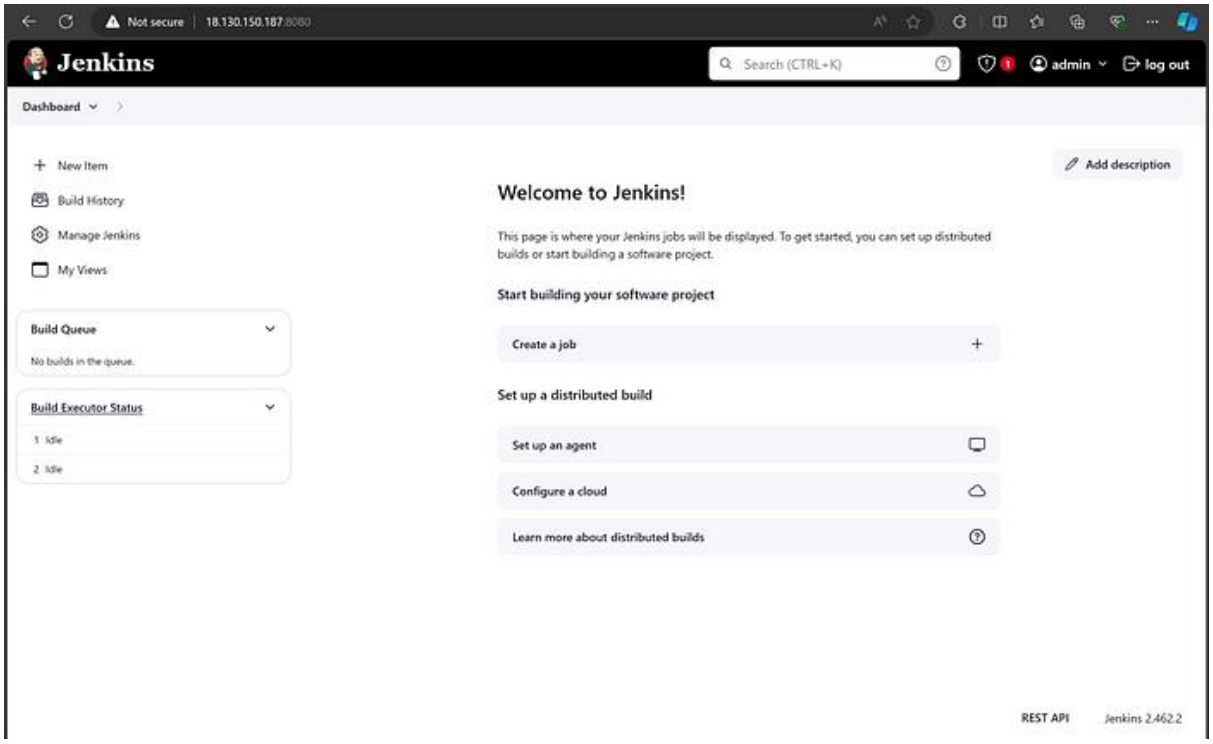


login to Jenkins



After running the sudo cat /var/lib/jenkins/secrets/InitialAdminPassword copy and paste the output to login to Jenkins. When you login click on suggested plugins this will help to download all the neccessary plugins plugins required for Jenkins to function. > Next, type in login details (you can use admin for both username and password and type in your email)

When this is done correctly we can now access the Jenkins page.



d.> Install Docker:

```
#!/bin/bash


# Update existing list of packages
sudo apt-get update


# Install prerequisite packages
sudo apt-get install -y \
   ca-certificates \
   curl \
   gnupg \
   lsb-release


# Add Docker official GPG key
sudo mkdir -m 0755 -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg


# Set up the Docker stable repository
```

```
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
  $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

# Update the package index again
sudo apt-get update

# Install Docker Engine, CLI, and containerd
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

# Start Docker
sudo systemctl start docker

# Enable Docker to start on boot
sudo systemctl enable docker

# Verify Docker installation
sudo docker --version

# Add current user to the Docker group to avoid using sudo (optional)
sudo usermod -aG docker $USER

echo "Docker installation completed. Please log out and log back in to apply the group changes."
```

To use the script: i> Copy the script to a file (e.g., install_docker.sh). ii.>Make the script executable: chmod +x install_docker.sh iii.>Run the script: ./install_docker.sh

This will install Docker and add your user to the Docker group to avoid using sudo for Docker commands.

```
ubuntu@ip-172-31-14-254:~$ vi install_docker.sh
ubuntu@ip-172-31-14-254:~$ chmod +x install_docker.sh
ubuntu@ip-172-31-14-254:~$ ./install_docker.sh
```

Check docker version by running the command docker --version

NOTE: This docker script will be used on all other VMs or Instances in this project. You can copy, paste and run the script to have docker installed on all the instances

3.2. Nexus set up

For Nexus: SSH into the VM > Run the command sudo apt update > Next, copy and paste the docker script used earlier and install the script.

When this is done. We will be running Nexus as a docker container

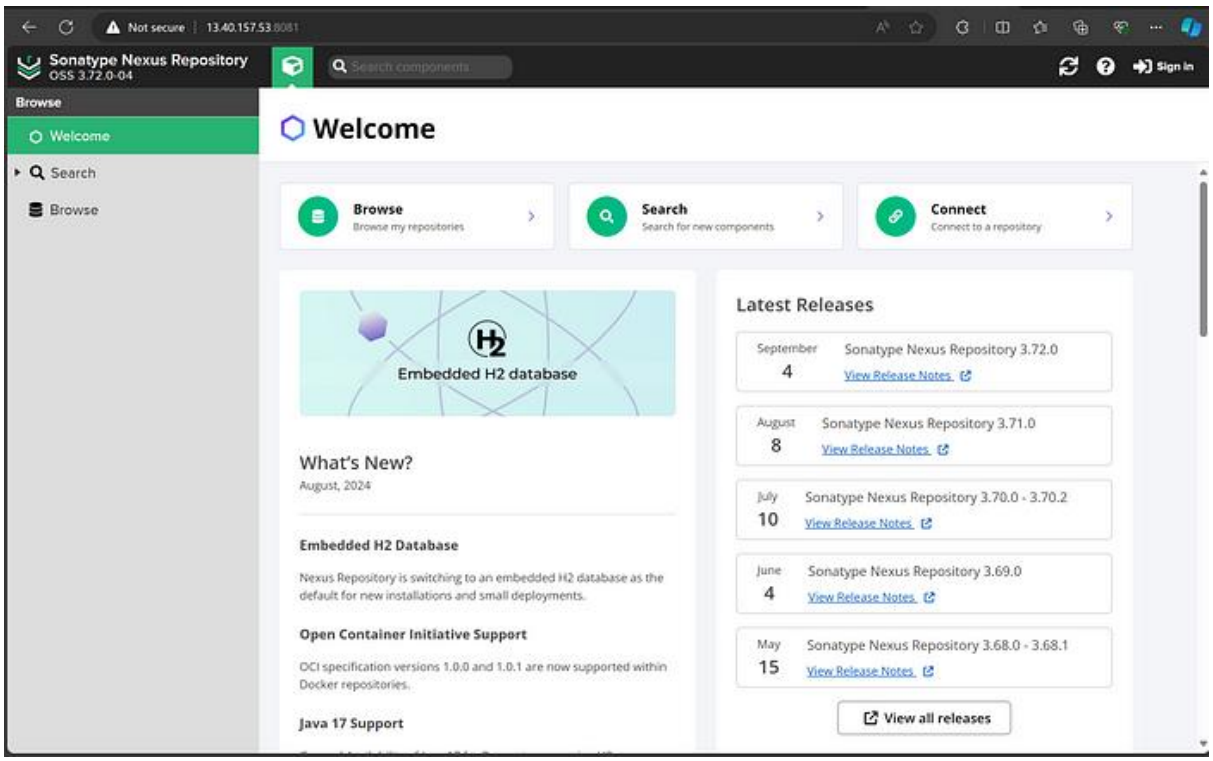
sudo docker run -d -p 8081:8081 sonatype/nexus3

```
ubuntu@ip-172-31-4-38:~$ sudo docker run -d -p 8081:8081 sonatype/nexus3
Unable to find image 'sonatype/nexus3:latest' locally
latest: Pulling from sonatype/nexus3
56f27190e824: Pull complete
aafeba3ef54d: Pull complete
bbd12c661c6b: Pull complete
8fa91fe19f70: Pull complete
b3575b896998: Pull complete
4bc21ba3b72a: Pull complete
6ffb1f80a976: Pull complete
Digest: sha256:c2c60fb29799b10e398615f6dcbf53f7e0a86650ee28f180585dacc3a7889a0d
Status: Downloaded newer image for sonatype/nexus3:latest
0f91fcdc8d5d15c481ad768052fa73937508e9bf2b8eb9d5a5a37b7f85719225
ubuntu@ip-172-31-4-38:~$
```

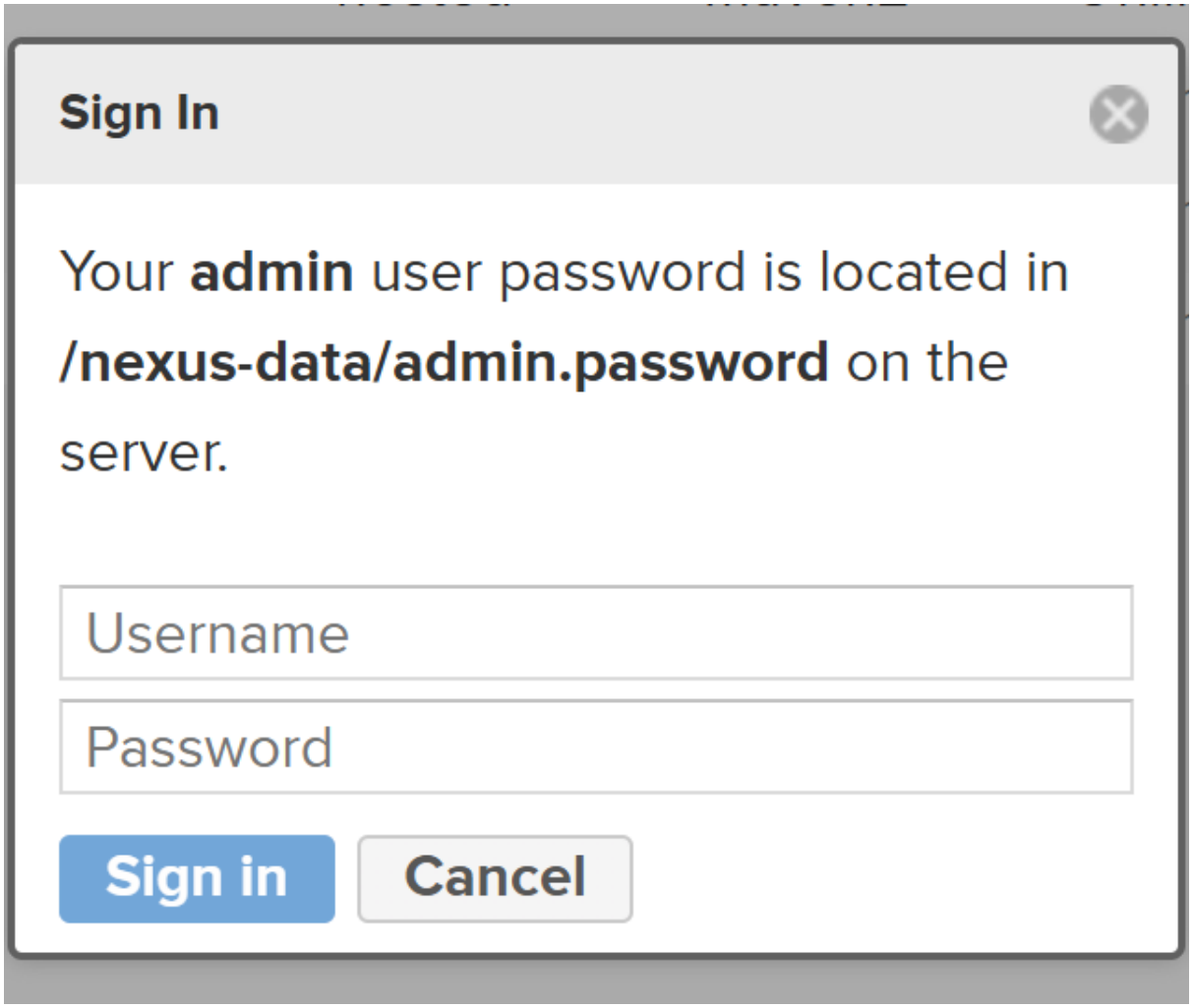
Confirm that the Nexus container is running sudo docker ps

```
ubuntu@ip-172-31-3-156:~$ sudo docker ps
CONTAINER ID   IMAGE            COMMAND                  CREATED          STATUS         PORTS                                       NAMES
98198fb5eac7   sonatype/nexus3  "/opt/sonatype/nexus…"   10 seconds ago   Up 7 seconds   0.0.0.0:8081->8081/tcp, :::8081->8081/tcp   serene_rhodes
ubuntu@ip-172-31-3-156:~$
```

We can access Nexus on http://<nexus-ip>:8081 > the page may take sometime to come up, just give it some time.



Sign in: To be able to login, click on the signin button and run this command on the terminal



sudo docker exec -it <sonar-docker-container-name> /bin/bash

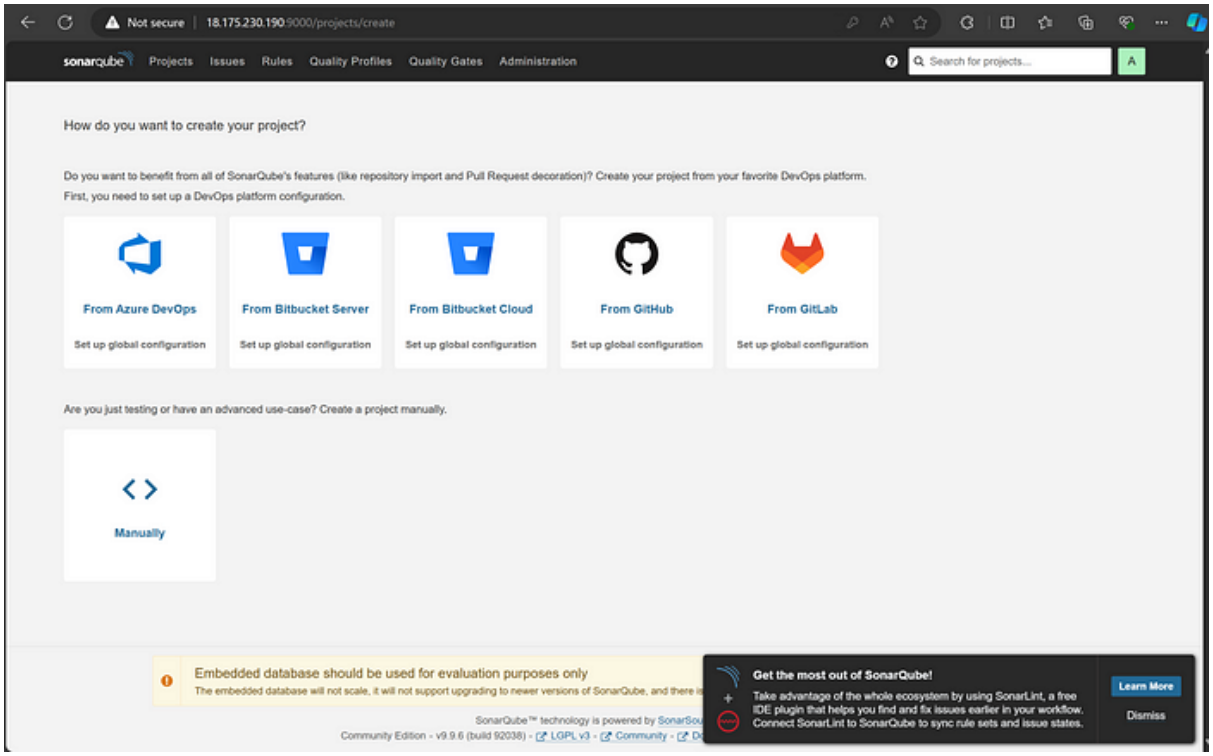
cat /nexus-data/admin.password

To login, use admin as username and the generated password. > Click on Accept Anonymous Access

3.3. Sonarqube setup

SSH into the sonarqube VM and update it > Install docker with the same script used earlier > check if docker is installed docker --version > Next, create a sonarqube container by running the command sudo docker run -d -p 9000:9000 sonarqube:lts-community > check the container is running sudo docker ps

Access sonarqube on http://<sonarqube-ip:9000 > login using admin as username and password



Step 4: Install Jenkins Plugins, and Configure Nexus, Trivy, SonarQube and DockerHub to use Jenkins

a.> Install Jenkins Plugin

In the Jenkins page, we need to install some additional plugins we need for this project.

on the left menu, click on 'manage jenkins' > click on plugins > available plugins > in the search bar; type and select the following plugins.

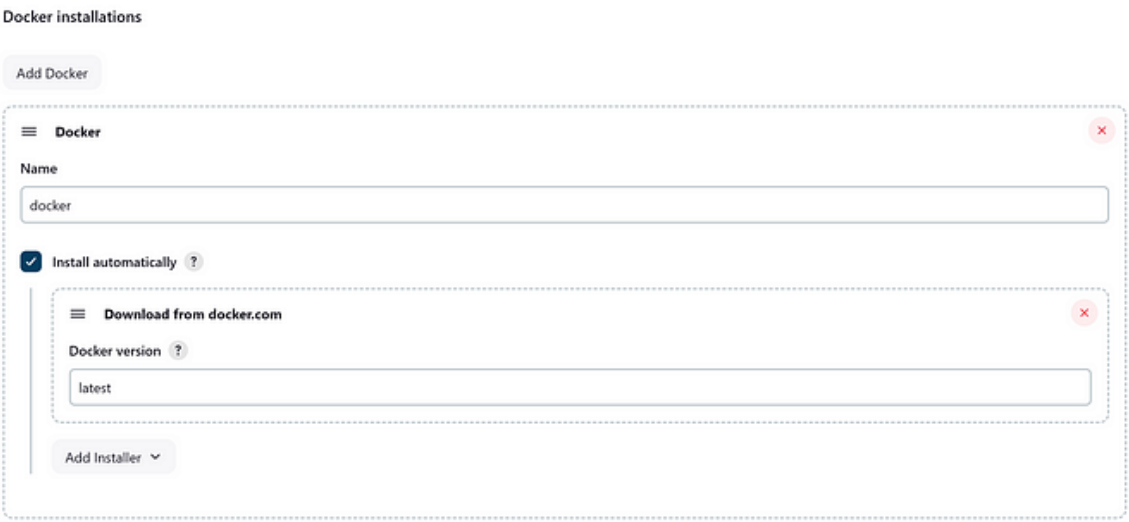
sonarqube scanner, eclipse temurin installer, config file provide, maven integration, pipeline maven integration, kubernetes, kubernetes credential, kubernetes CLI, kubernetes client API, docker, docker pipeline

>After selecting them, click on install > Restart Jenkins when the installation is done.

b.> Configure the Plugins:

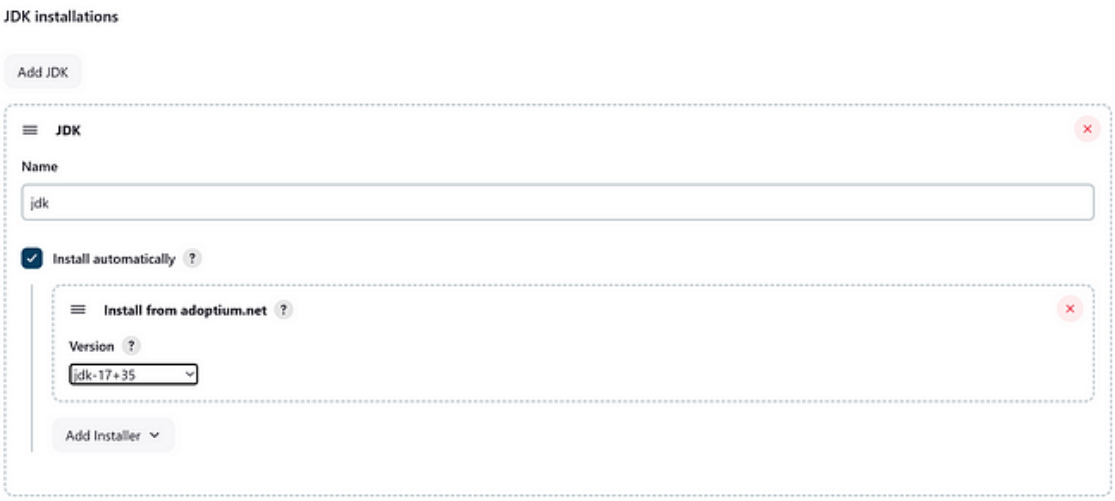
When you install plugins, it is a good practice to configure based on your needs. To do this; click on 'manage jenkins' > click on 'tools'

For docker: give a name and leave as default

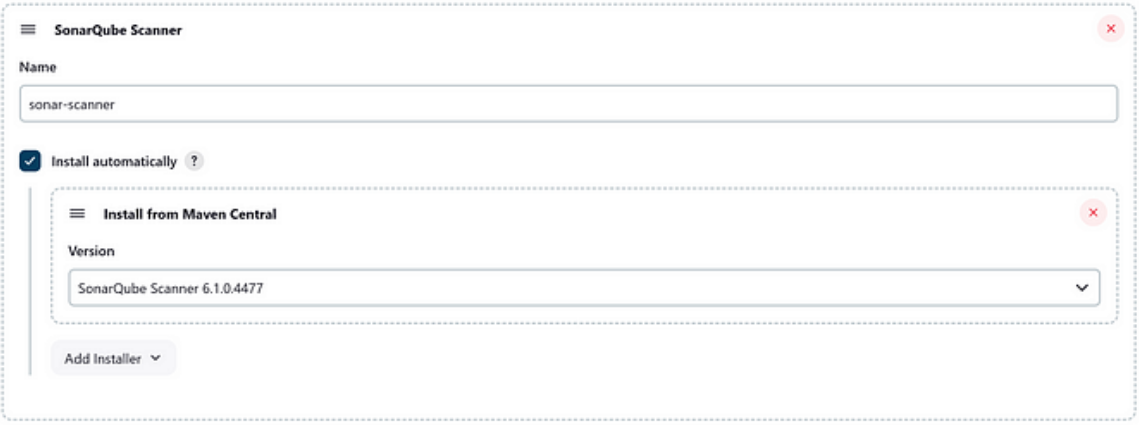


For maven: give it a name and leave as default

For JDK: click on add > install automatically > click on 'adoptium.net' > select jdk 17+35 > Save the configuration



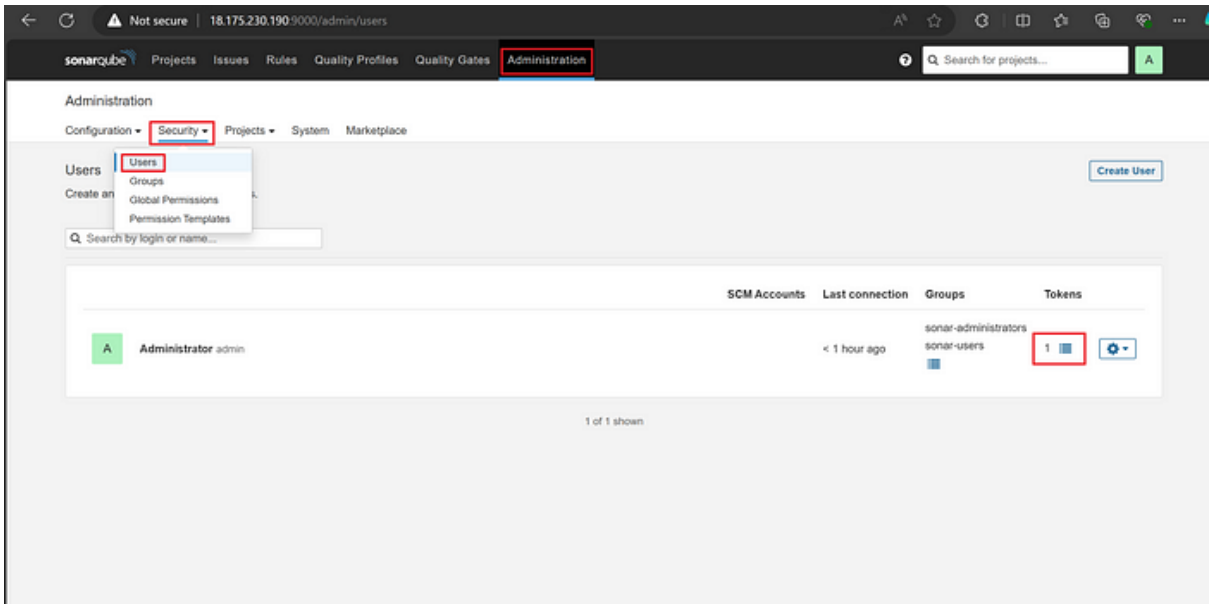
for sonarqube scanner

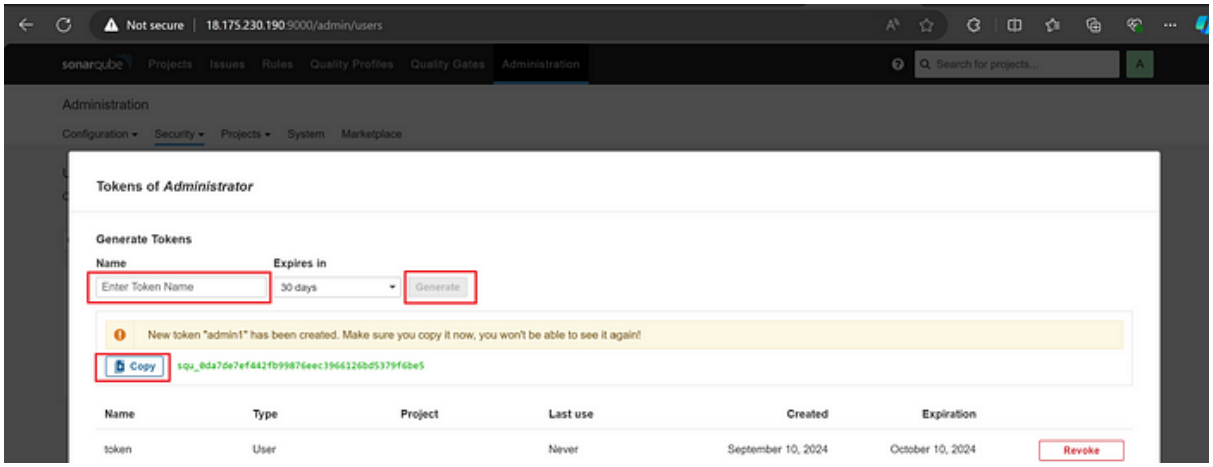


c.> Configure SonarQube Scanner and SonarQube Server

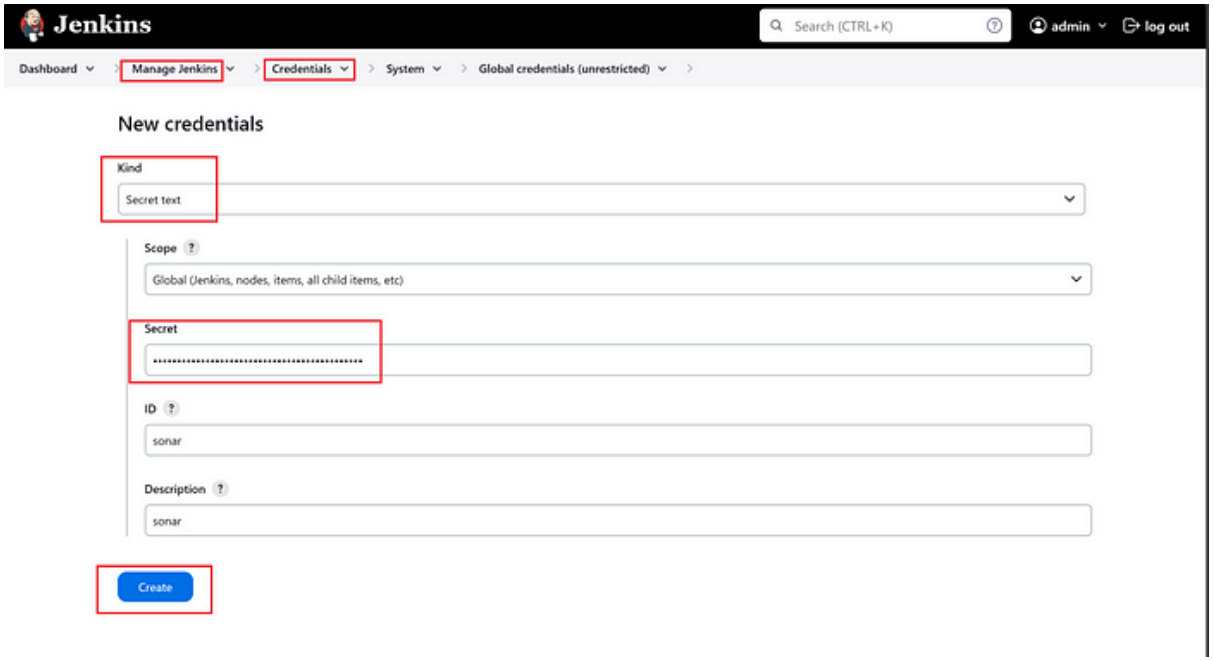
- Generate token for SonarQube Scanner**:**

To generate a token; at the top menu, click on ‘administration’ > security > in the dropdown menu, click on ‘users’ > then ‘Tokens’ > type in a name and generate the token > copy the token generated





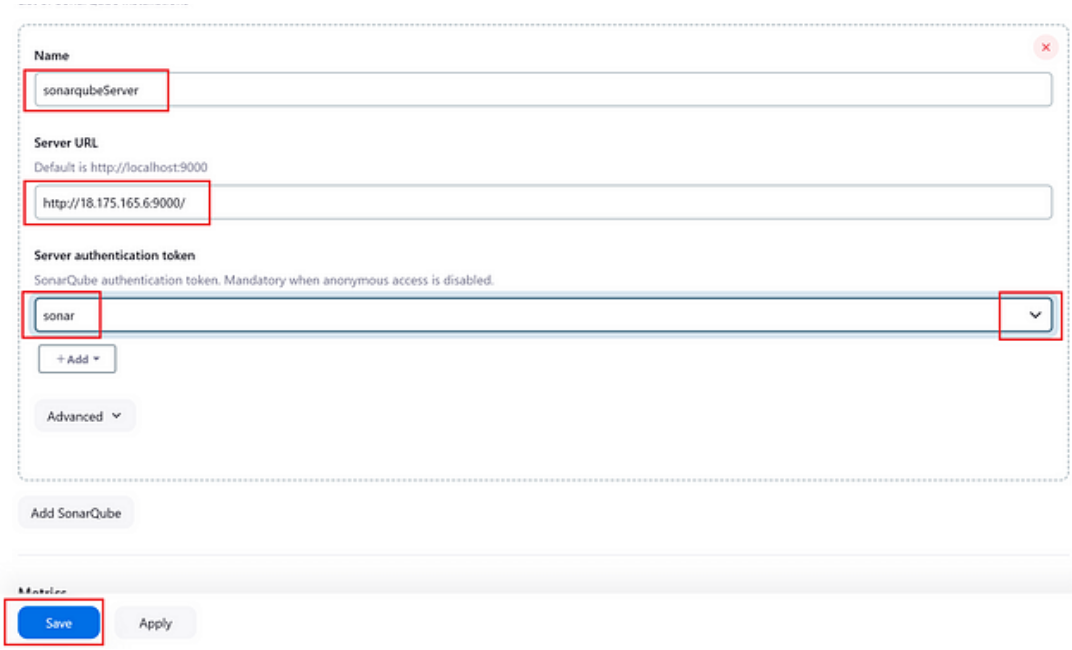
After generating the token we need to add this token to our jenkins credentials, login to your jenkins , click on ‘Manage Jenkins’ > Credentials > Global > click on ‘Add Credentials’ > in the pop page, for ‘kind’ select Secret text > paste the SonarQube token into the secret box > Create



- Add SonarQube Server Credentials to Jenkins

In 'Managed Jenkins' > Click on System > under SonarQube Servers click on Add SonarQube

> for URL type in http://<your-sonarqube-ip:9000 >



d.> Configure Jenkins to use Trivy

In the jenkins, click on 'New Items' > Select 'pipeline' > give a name for the pipeline > and create it. > when it is created, select the pipeline to start building. > click on 'configure'

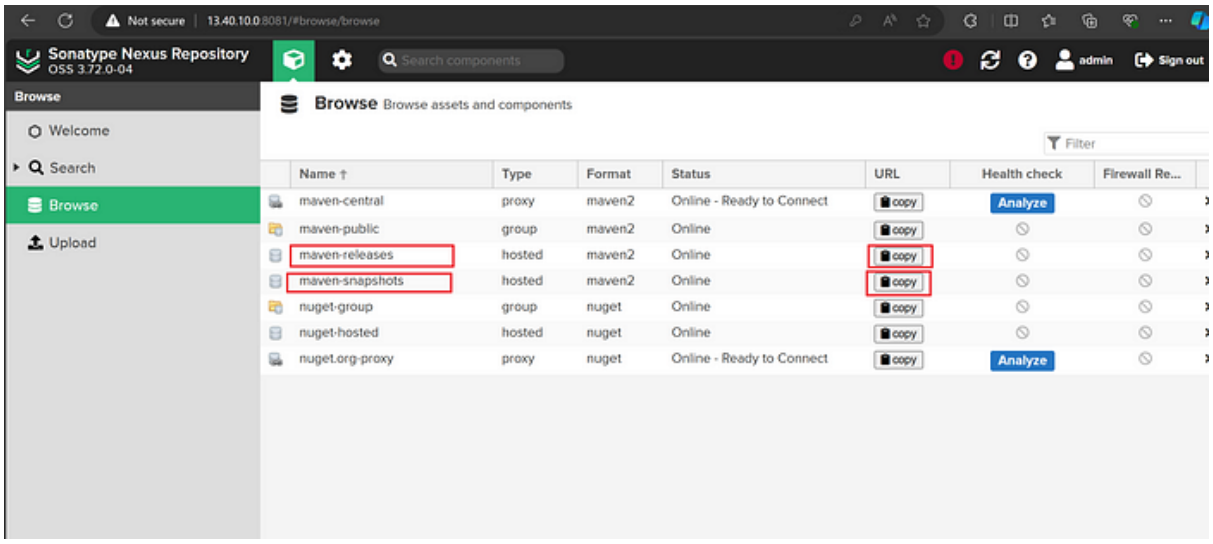
In the jenkins VM terminal, run the following commands to install trivy plugins . This is important if not our trivy pipeline commands will not run.

wget https://github.com/aquasecurity/trivy/releases/download/v0.43.0/trivy_0.43.0_Linux-64bit.deb

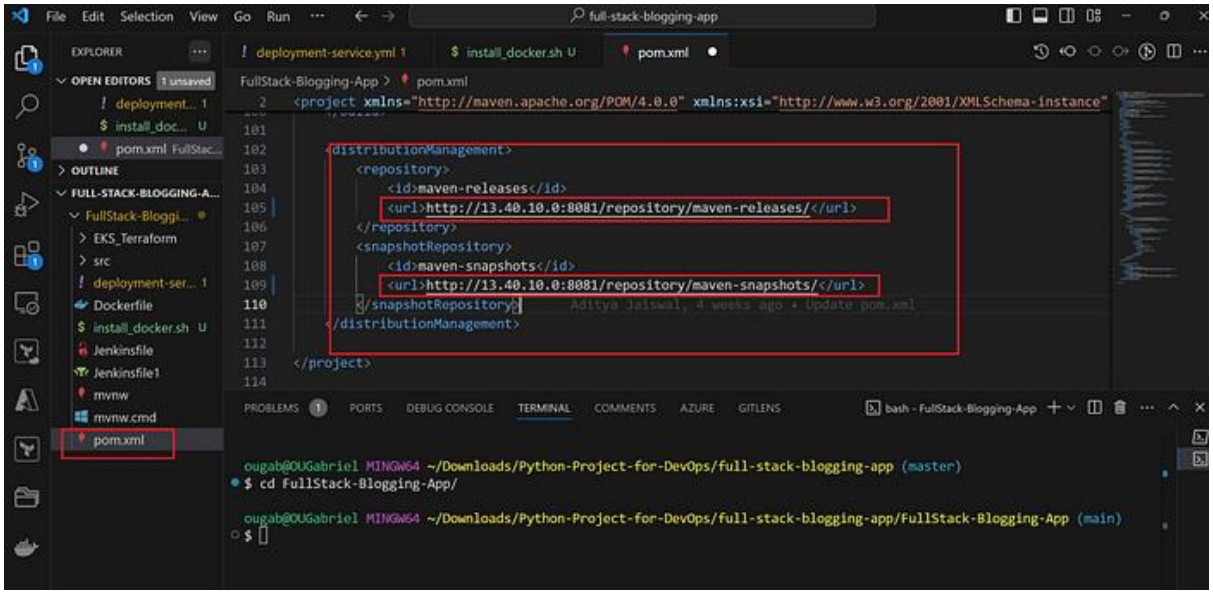
sudo dpkg -i trivy_0.43.0_Linux-64bit.deb

e.> Configure Jenkins to use Nexus

> copy th maven-releases and maven-snapshotsURL



Depending on where you have your source code open, edit the pom.xml file and paste it there. (if you have the source code on vscode like I did, make sure to push the changes to github)



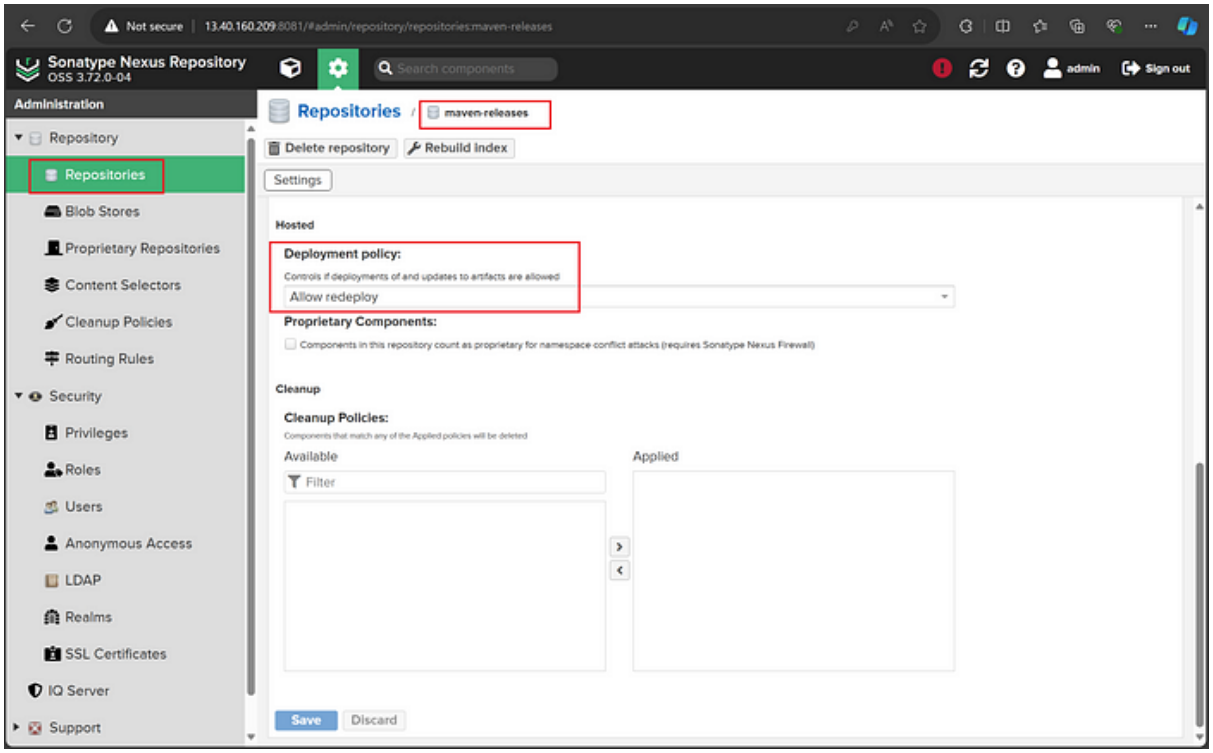
To complete the setup, Jenkins needs authentication to the Nexus Server.

go to Managed Jenkins > Managed Files > Add ‘A new config file’ > Select ‘Global Maven settings.xml’ > scroll down, for IDchange it to ‘maven-settings’ or any name you can remember > In the pop page, edit the server section > Change username and password with your Nexus login credentials . (*In the server section make sure by removing the comments and making sure it looks like this.*)

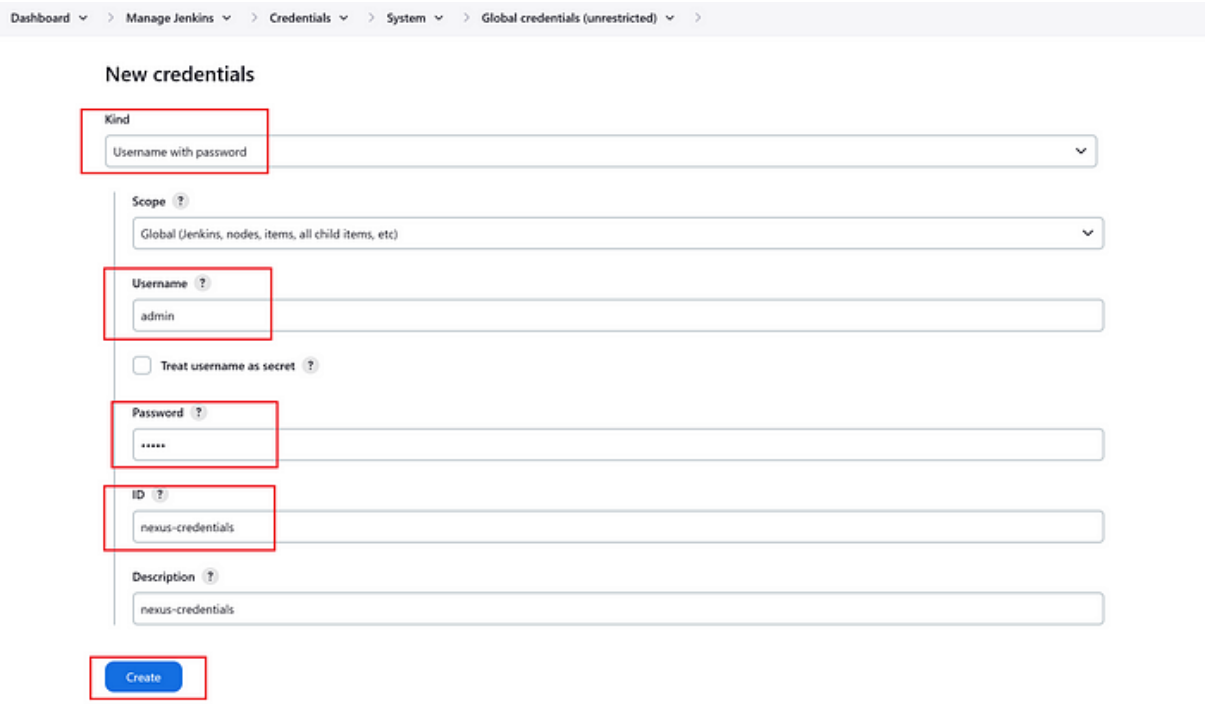
```
 <servers>


   <server>

     <id>maven-releases</id>

     <username>admin</username>

     <password>admin</password>

   </server>


   <server>

     <id>maven-snapshots</id>

     <username>admin</username>

     <password>admin</password>

   </server>


  </servers>
```

in the Nexus login page, edit the deployment policy for both maven-releases and maven-snapshots > change it to allow redeploy

This will ensure there is no re-build error when the jenkins pipeline attempts to re-send the artifacts



Add credentials to jenkins, in managed jenkins > credentials > global > type in the username and password of the Nexus credentials and create it

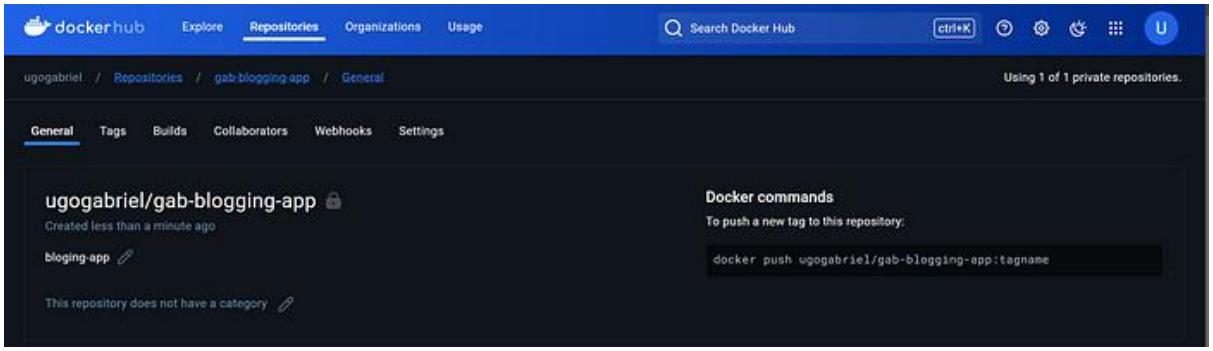


rf.> Configure Jenkins to Dockerhub authentication

Before building the artifact, we need a registry to store the image. We will doing this with dockerhub

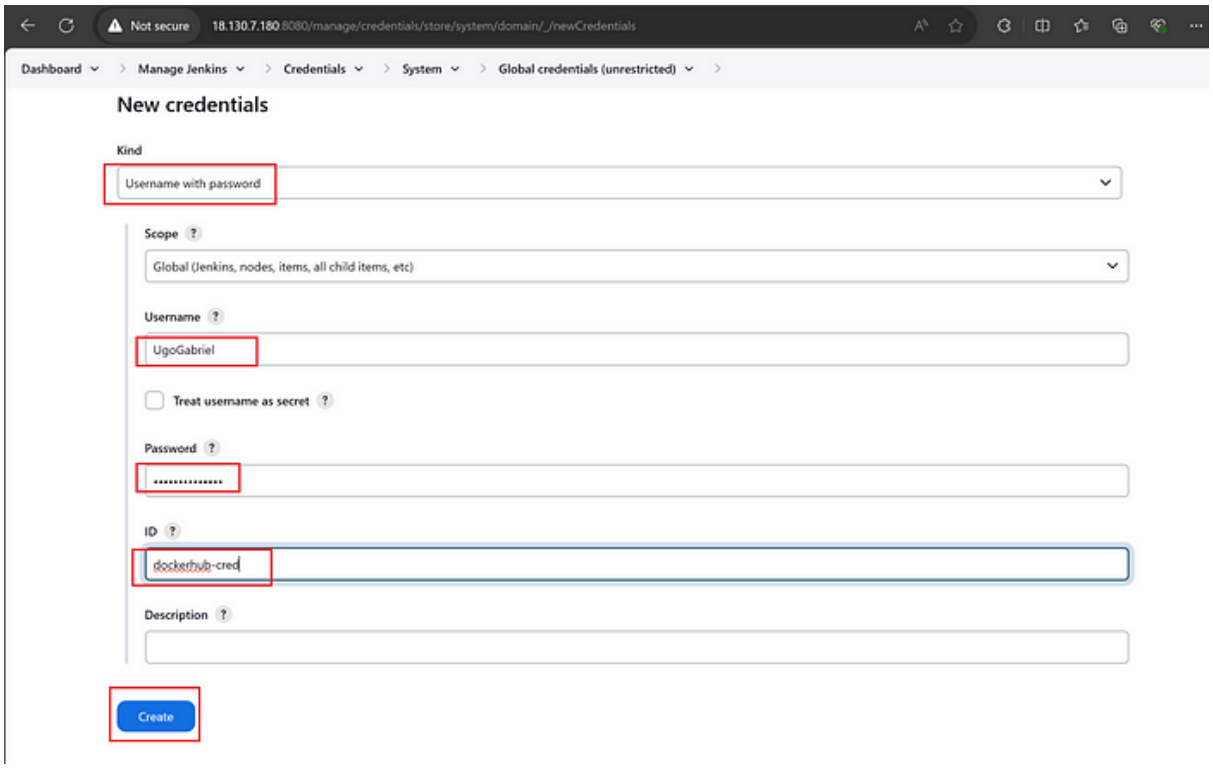
> Log into your dockerhub and create a **private** repo, you can give it any name .

We will be adding this newly created dockerhub repo into the Docker Build & Tagstage of the pipeline



After creating the dockerhub repo, we need to create a jenkins credential for it.

Go to 'managed jenkins' > click on 'credentials' > > global > then 'Add credentials' > for kind, select 'username and password' > type in your dockerhub username and password in their respective fields > for ID give it a name so you can can identify it.



We will need to add the credentials to system settings, go to managed jenkins > systems > edit the docker section and select the newly docker credentials you just created.

Step 5: Create a complete CICD pipeline

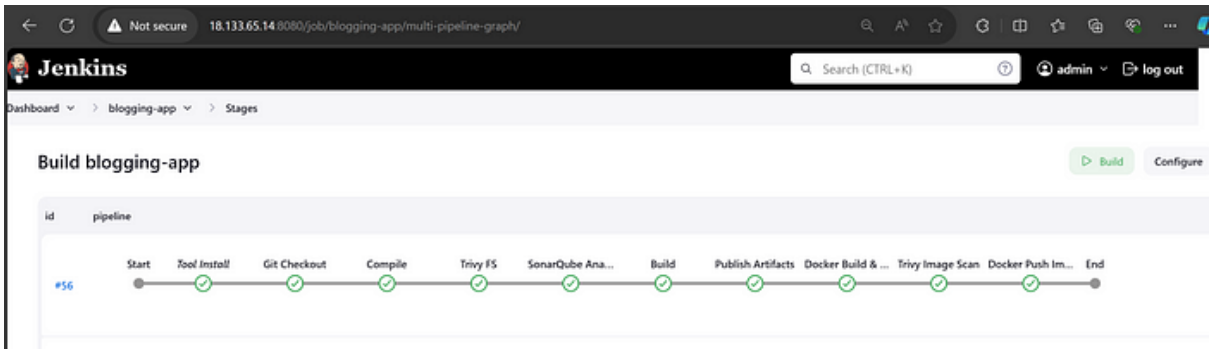
This pipeline should be changed to fit your docker image and dockerhub details. This is not the end of the pipeline, because we still going to integrate Email Push Notification within the pipeline that tells us when the pipeline fails or succeeds.

```
pipeline {
  agent any
  tools {
    jdk "jdk"
    maven "maven"
  }
  environment {
    SCANNER_HOME = tool 'sonar-scanner'
  }

  stages {
    stage('Git Checkout') {
      steps {
        git branch: 'main', url: 'https://github.com/ougabriel/full-stack-blogging-app.git'
      }
    }
    stage('Compile') {
      steps {
        sh "mvn compile"
      }
    }
    stage('Trivy FS') {
      steps {
        sh "trivy fs . --format table -o fs.html"
      }
    }
    stage('SonarQube Analysis') {
      steps {
```

```
            withSonarQubeEnv('sonarqubeServer') {
                sh '''$SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=Blogging-app -Dsonar.projectKey=Blogging-app \
                    -Dsonar.java.binaries=target'''
            }
        }
    }
    stage('Build') {
        steps {
            sh "mvn package"
        }
    }
    stage('Publish Artifacts') {
        steps {
            withMaven(globalMavenSettingsConfig: 'maven-settings', jdk: 'jdk', maven: 'maven', mavenSettingsConfig: '', traceability: true) {
                sh "mvn deploy"
            }
        }
    }
    stage('Docker Build & Tag') {
        steps {
            script{
            withDockerRegistry(credentialsId: 'dockerhub-cred', url: 'https://index.docker.io/v1/') {
            sh "docker build -t ugogabriel/gab-blogging-app ."
            }
            }
        }
    }
    stage('Trivy Image Scan') {
        steps {
            sh "trivy image --format table -o image.html ugogabriel/gab-blogging-app:latest"
```

```
        }
      }
      stage('Docker Push Image') {
        steps {
          script{
          withDockerRegistry(credentialsId: 'dockerhub-cred', url: 'https://index.docker.io/v1/') {
            sh "docker push ugogabriel/gab-blogging-app"
          }
          }
        }
      }
    } // Closing stages
} // Closing pipeline
```

Run the build, and click on stages to see the pipeline stages



Step 6: Create the EKS cluster, Install AWS CLI, Kubectl and Terraform

We will need to create a VM, install and useterraform to deploy the EKS service into the machine and install and usekubectl to interact with this EKS cluster.

a.> Create the VM: Login to the AWS console and create a new EC2 instance (t2 medium, 15GB) as we did before > use the same keypair and security group as the other instances and make sure the public IP is enabled. > Create the instance.

b.> Install AWS CLI: The AWS CLI (Command Line Interface) allows you to interact with AWS services directly from your terminal.

```
# Update the package list

sudo apt update


# Install curl if not already installed
```

sudo apt install curl -y

Download the AWS CLI v2 installation file

curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"

Install unzip and extract the downloaded zip file

sudo apt install unzip -y

unzip awscliv2.zip

Run the AWS CLI installer

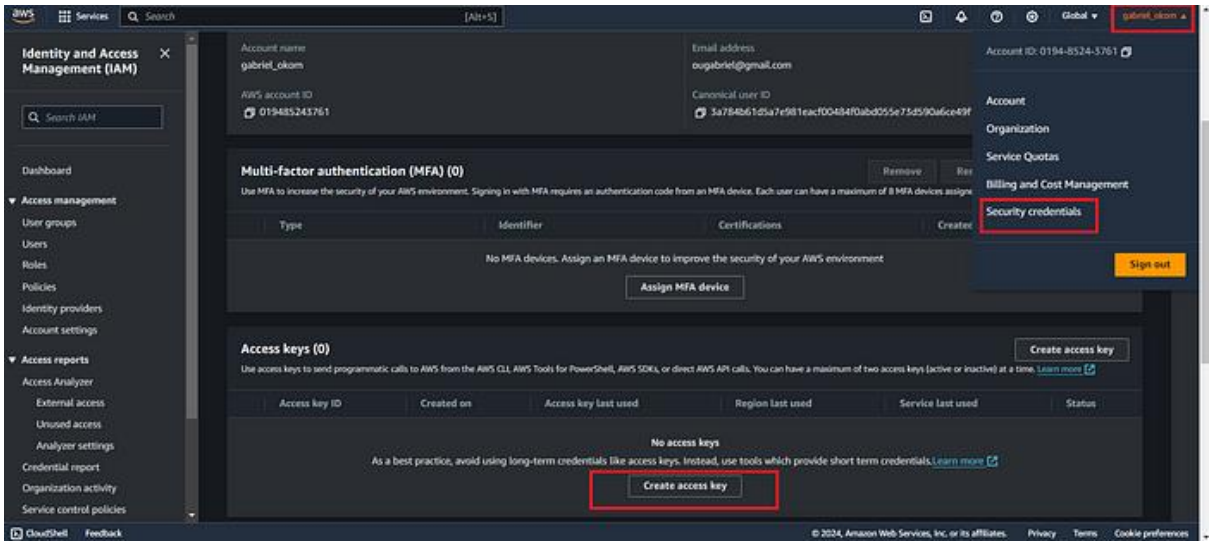
sudo ./aws/install

Verify the installation

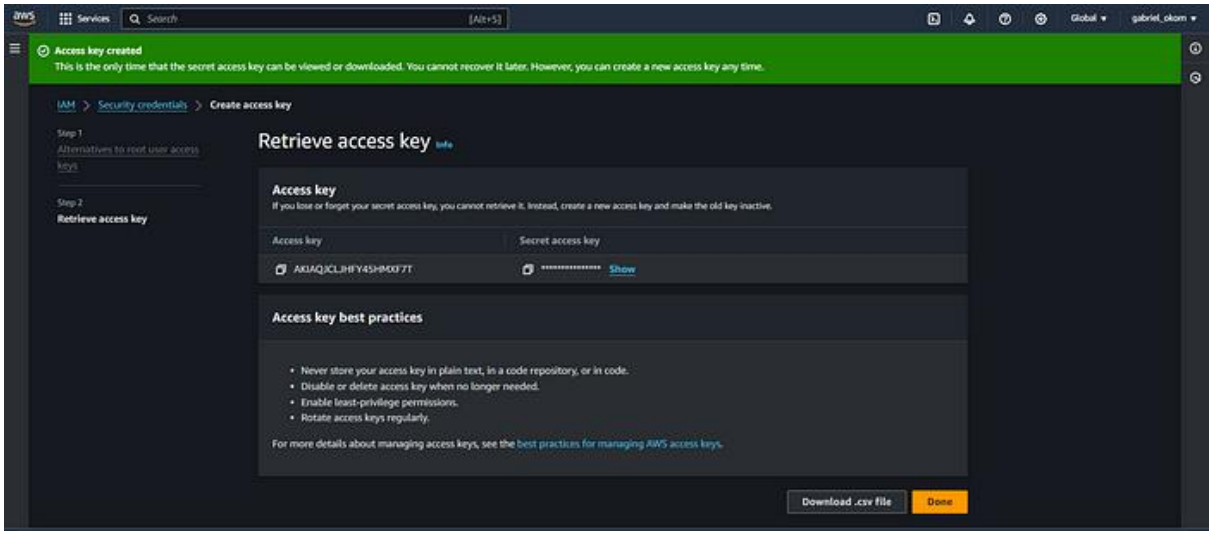
aws --version

After the installation we need to connect the cluster to our AWS; to do this we need to create a access key that will be to authenticate to AWS services.

> In your AWS console > click on your profile > security credentials > and Create access key



Run the following command aws configurein your AWS console > copy and paste the access key and security access keys in the prompts.



Follow the prompts as shown below

```
ubuntu@ip-172-31-25-178:~$ aws configure
AWS Access Key ID [None]: AKIAQJCLJHFY45HMXF7T
AWS Secret Access Key [None]: 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
Default region name [None]: eu-west-2
Default output format [None]:
ubuntu@ip-172-31-25-178:~$
```

c.> Install Terraform: Install terraform in this instance by running this command

sudo apt install terraform --classic

We need 3 tf files for our terraform script. Which is the main.tf , output.tf and variable.tf . ***(find the files in the given github repo for this project)***

vi output.tf

vi main.tf

vi variable.tf

Run this command individually, copy and paste the terraform scripts into the editor > save and exit.

```
ubuntu@ip-172-31-25-178:~$ ls
main.tf  output.tf  variable.tf
ubuntu@ip-172-31-25-178:~$
```

To deploy the resources, we need to first run the following commands

(optional: make sure to change the main.tf ***region and the availabilty zone to suite your region)***

- **Important**: in the variable.tfscript you MUST change the default = "gabkeypair to the name of your AWS instance keypairif the name is different from this.
- terraform init : to initialize the project, It downloads the necessary provider plugins and sets up the backend where Terraform will store state data.
- terraform plan : Prepares an execution plan, showing what actions Terraform will take to deploy the infrastructure. It lists the resources that will be created, modified, or destroyed. In this case, you would see a plan indicating that **17 resources** are going to be deployed.

```
Plan: 17 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + cluster_id     = (known after apply)
  + node_group_id = (known after apply)
  + subnet_ids     = [
      + (known after apply),
      + (known after apply),
    ]
  + vpc_id         = (known after apply)

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.
ubuntu@ip-172-31-25-178:~$
```

- terraform apply: Executes the plan and actually deploys the infrastructure. Terraform will create, modify, or delete resources as outlined in the execution plan.

preferrably; run this command

terraform apply --auto-approve

Make sure to run the commands in the order given above

When you run the command, it will take sometime for all the changes or all the services to be fully deployed.

After the installation when you run kubectl get nodes you will notice the error Command kubectl not found this is because the kubectl command is not yet installed.

sudo snap install kubectl --classic

```
ubuntu@ip-172-31-14-254:~$ sudo snap install kubectl --classic
kubectl 1.31.1 from Canonical✓ installed
ubuntu@ip-172-31-14-254:~$
```

After installing kubectl we need to confirm if our nodes are ready; run the command kubectl get nodes . You should get another error as shown below.

```
ubuntu@ip-172-31-25-178:~$ kubectl get nodes
E0916 22:20:06.276987   24856 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": dial tcp 127
.0.0.1:8080: connect: connection refused"
E0916 22:20:06.278538   24856 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": dial tcp 127
.0.0.1:8080: connect: connection refused"
E0916 22:20:06.279789   24856 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": dial tcp 127
.0.0.1:8080: connect: connection refused"
E0916 22:20:06.281422   24856 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": dial tcp 127
.0.0.1:8080: connect: connection refused"
E0916 22:20:06.282861   24856 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": dial tcp 127
.0.0.1:8080: connect: connection refused"
The connection to the server localhost:8080 was refused - did you specify the right host or port?
ubuntu@ip-172-31-25-178:~$
```

This is because you are yet to connect the EKS cluster to AWS services, do that using this command;

aws eks --region eu-west-2 update-kubeconfig --name <cluster-name>

Run the command again this time to the nodes ready

```
ubuntu@ip-172-31-25-178:~$ aws eks --region eu-west-2 update-kubeconfig --name devopsshack-cluster
Added new context arn:aws:eks:eu-west-2:019485243761:cluster/devopsshack-cluster to /home/ubuntu/.kube/config
ubuntu@ip-172-31-25-178:~$ kubectl get nodes
NAME                                        STATUS   ROLES    AGE   VERSION
ip-10-0-0-208.eu-west-2.compute.internal    Ready    <none>   15m   v1.30.2-eks-1552ad0
ip-10-0-1-200.eu-west-2.compute.internal    Ready    <none>   14m   v1.30.2-eks-1552ad0
ip-10-0-1-47.eu-west-2.compute.internal     Ready    <none>   14m   v1.30.2-eks-1552ad0
ubuntu@ip-172-31-25-178:~$
```

d.> Setup Service Account and RBAC

RBAC is one of the most important concepts of Kubernetes. In order to be able to perform deployments and authentication with this cluster we need to create a service account and give it necessary permissions. RBAC (Role Based Access Control) we will create roles, rolebinding, a token for the secret which we will be used for authentication. Copy and paste the following commands into the vi editor

Create a namespace

```
kubectl create namespace webapps
Service Account > vi serviceaccount.yaml
apiVersion: v1
kind: ServiceAccount
metadata:
  name: jenkins
  namespace: webapps
Role > vi role.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: webapps
  name: role
rules:
- apiGroups: ["*"]
  resources: ["*"]
  verbs: ["get", "list", "create", "delete", "patch", "watch"]
Rolebinding > vi rolebinding.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: rolebinding
  namespace: webapps
subjects:
- kind: ServiceAccount
  name: jenkins # The service account created earlier
  namespace: webapps
roleRef:
  kind: Role
  name: role  # The role created earlier
  apiGroup: rbac.authorization.k8s.io
```

Token for Service Account Secret > vi sa-secret.yaml

apiVersion: v1

kind: Secret

type: kubernetes.io/service-account-token

metadata:

  name: mysecretname

  namespace: webapps # Make sure to specify the correct namespace

  annotations:

    kubernetes.io/service-account.name: jenkins # The service account name

Apply all the yaml files,

kubectl apply -f serviceaccount.yaml

kubectl apply -f role.yaml

kubectl apply -f rolebinding.yaml

kubectl apply -f sa-secret.yaml

If you check the git repo for this project, you will see the deployment-service.yml which will be deployed to this EKS cluster. We will need to create an imagePullSecretfor it to pull our docker image from the dockerhub private registry . When working with private registry, you will need to provide an authentication secret that helps it get access to the private registry. Run the following command. (change username and password to that of your dockerhub credentials).

kubectl create secret docker—registry regcred \

--docker—server=https://index.docker.io/v1/ \

--docker—username=<your—username> \

--docker—password=<your—password>

--namespace=webapps

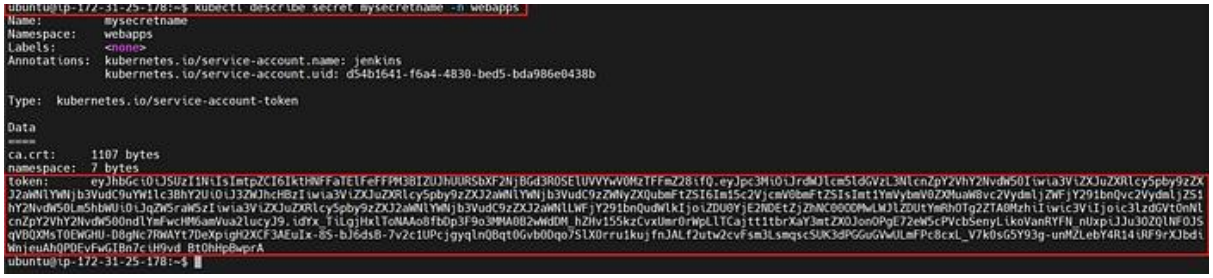
regcred *is the name of the* imagePullSecret *. This is already added to the* deployment.yaml *file in the git repo.*

e.> Authenticate Jenkins with EKS using secret

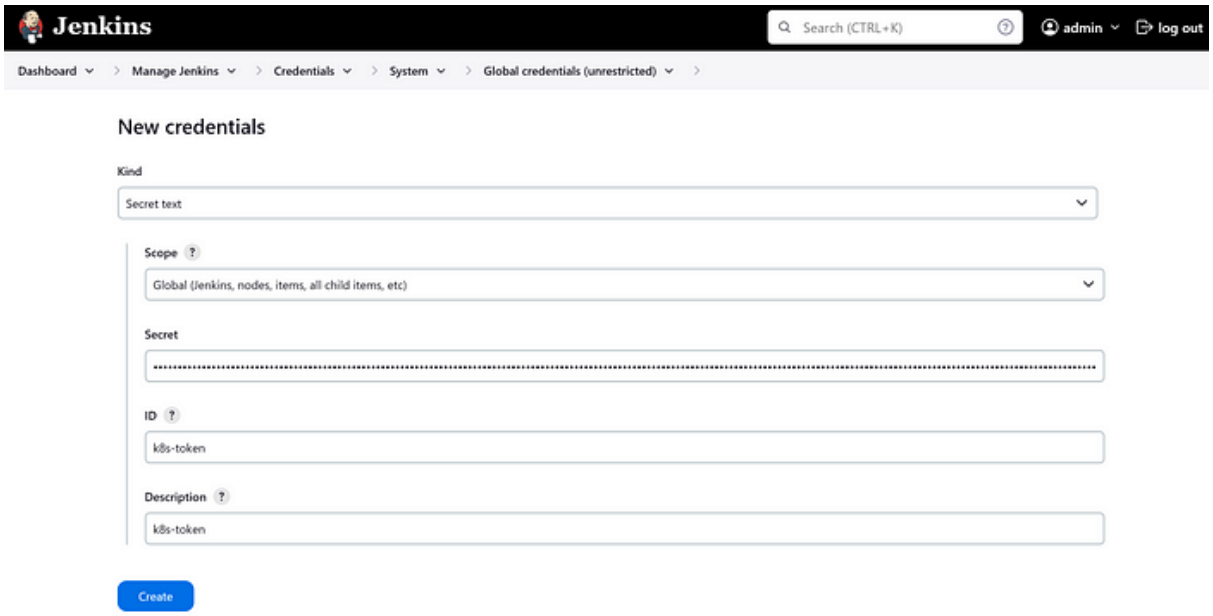
If you run the command, kubectl get secrets -n webapps you will notice we have two secrets.

```
ubuntu@ip-172-31-25-178:~$ kubectl get secrets -n webapps
NAME           TYPE                                  DATA   AGE
mysecretname   kubernetes.io/service-account-token   3      20s
regcred        kubernetes.io/dockerconfigjson        1      10m
ubuntu@ip-172-31-25-178:~$
```

One for pulling the image from the dockehub private registry and the other for the service account authentication. Run the command kubectl describe secret mysecretname -n webapps to get the authentication token for our service account secret.



Go to jenkins, click on ‘manage jenkins’ > Credentials > Global > Add credentials > for kind select ‘secret text’ > paste the token within the ‘secret’ box > give it a name > Create



f.> Install Kubectl on Jenkins, Modify the Pipeline and Setup Email Notification

We will be inserting kubectl command in the pipeline, we will need to install it on the Jenkins machine so that kubectl command syntax can be able to work.

sudo snap install kubectl --classic

Now, we will be modifying the pipeline to deploy our kubernetes resources. we will be adding 3 more stages to the pipeline script for this purpose.

g.> Setup Email Notifications

Copy and paste the following into the browser to generate an email authentication password. (GMAIL only)

https://myaccount.google.com/apppasswords

for other email accounts, you can do this.

For Hotmail/OutlookGo to your Microsoft Account Security page.

Click on Advanced security options.

Under App passwords, click on Create a new app password.

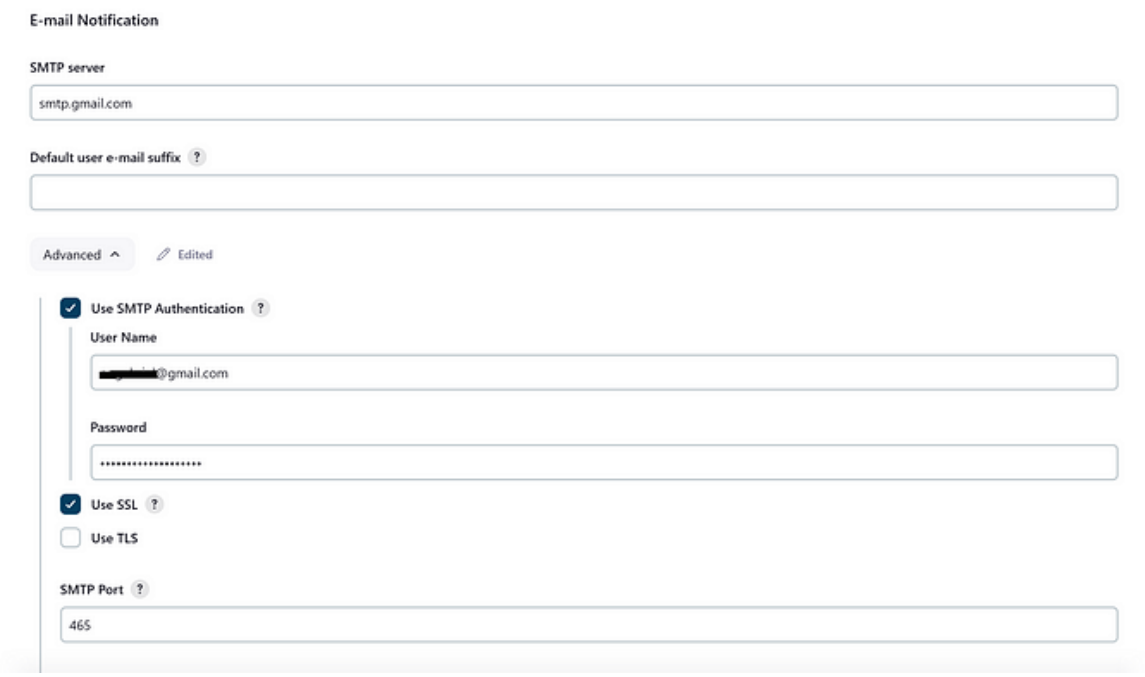
Use the generated app password in your application (e.g., Jenkins) instead of your regular password.For Yahoo:Go to your Yahoo Account Security page.

Enable Two-step verification if not already enabled.

After enabling 2FA, select Generate app passwords.

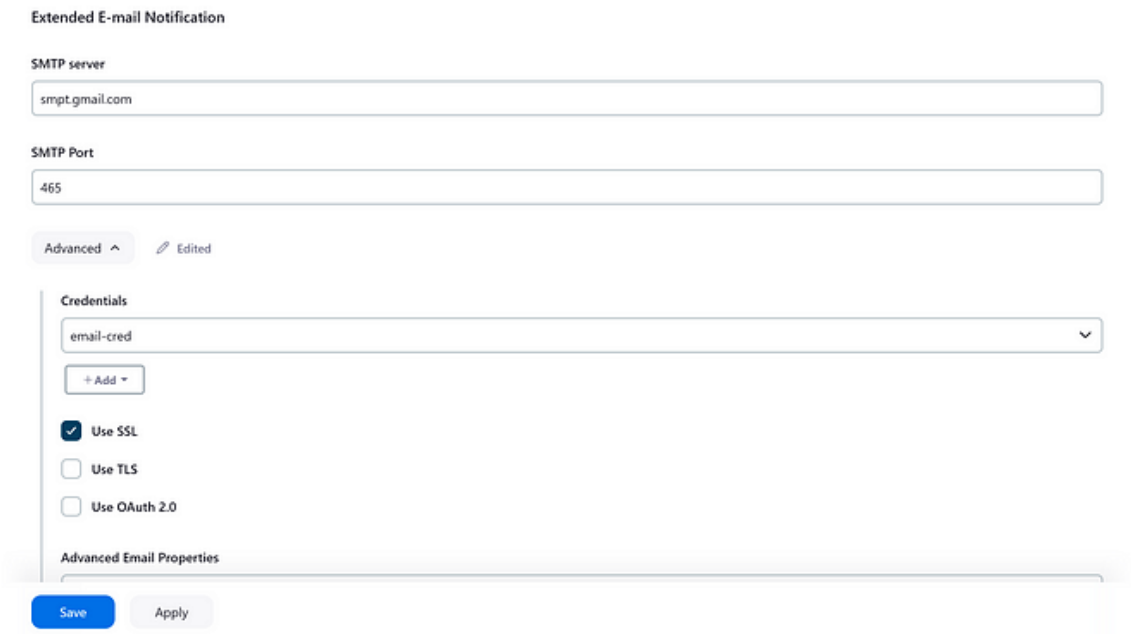
Use the app password in your application (e.g., Jenkins) instead of your regular password.

Click on 'manage jenkins' > systems > scroll down till you find Email Notification > fill in the boxes as shown > test the connection



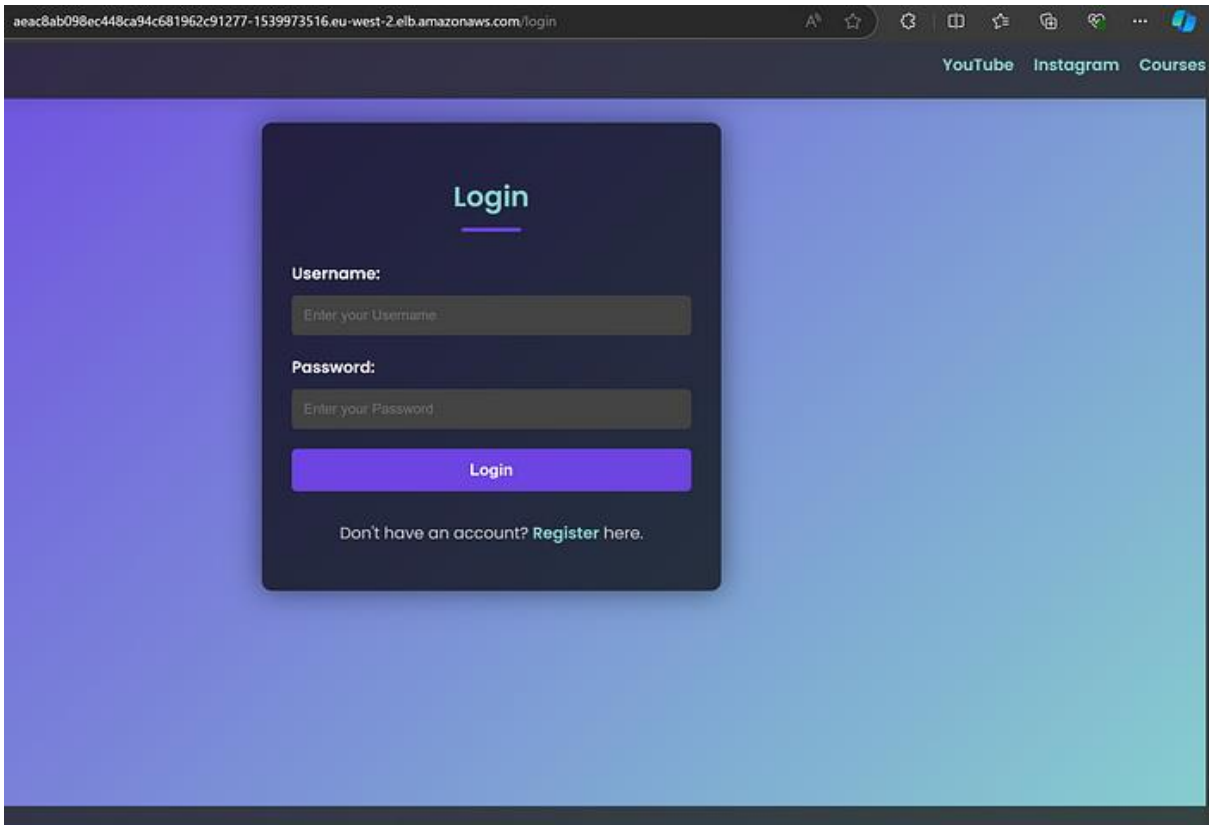
Make sure port 465 is open in your AWS NSG (Network Security Group).

Within the same page, configure the same thing for Extended Email notification , this time you will add a credential named email cred containing the username and generated password of the email you used earlier.



To see the 2 new stages (k8s-deploy and k8s verify) added to the pipeline please check the [jenkinsfile in the git repo](). Trigger the pipeline to deploy the application

Step 7: Assign a custom domain to the deployed application



```
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] withKubeCredentials
[Pipeline] {
[Pipeline] sh
+ kubectl get pods
NAME                                    READY   STATUS    RESTARTS   AGE
bloggingapp-deployment-d46c9c7fb-5n7f4   1/1     Running   0          11m
bloggingapp-deployment-d46c9c7fb-gt5qv   1/1     Running   0          11m
[Pipeline] sh
+ kubectl get service
NAME                TYPE           CLUSTER-IP       EXTERNAL-IP                                                              PORT(S)
AGE
bloggingapp-ssvc    LoadBalancer   172.20.230.165   aeac8ab098ec448ca94c681962c91277-1539973516.eu-west-2.elb.amazonaws.com
80:30380/TCP   34m
[Pipeline] }
[kubernetes-cli] kubectl configuration cleaned up
[Pipeline] // withKubeCredentials
```

Add the URL link into an existing Domain from any domain name provider for example (godaddy) using the CNAME type. Perform an nslookup to verify it is up. Then try it on the browser to view the app.

Note: this is optional. I dont need this app to have any special domain url but you can try it for practice

Step 8: Monitor the application

Create another EC2 instance of t2large and 25GB > install grafana, prometheus and blackbox on the instance using the following command.

a.> Set Up Blackbox Exporter: Blackbox Exporter is used for probing endpoints and checking their availability.

Install Blackbox Exporter

Download and run Blackbox Exporter:

wget https://github.com/prometheus/blackbox_exporter/releases/download/v0.21.0/blackbox_exporter-0.21.0.linux-amd64.tar.gz

tar xvfz blackbox_exporter-0.21.0.linux-amd64.tar.gz

cd blackbox_exporter-0.21.0.linux-amd64

./blackbox_exporter

Configure Blackbox Exporter > vi blackbox.yaml

```
modules:
  http_2xx:
    prober: http
    timeout: 5s
    http:
      method: GET
```

```
    valid_http_versions: [ "1" ]
    valid_http_mimes: [ "application/json" ]
    valid_http_status_codes: []  # Defaults to 2xx
```

b.> Set Up Prometheus: Prometheus will be used to collect metrics from various sources, including the Blackbox Exporter.

Install Prometheus > sudo apt install prometheus -y

Create the Prometheus config file > vi prometheus.yaml

```
global:
  scrape_interval: 15s
  evaluation_interval: 15s


crape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']


  - job_name: 'blackbox'
    metrics_path: /probe
    params:
      module: [http_2xx]  # Look for an HTTP 200 response
    static_configs:
      - targets:
          - http://prometheus.io
          - https://prometheus.io
######blog app url link
          - aeac8ab098ec448ca94c681962c91277-1539973516.eu-west-2.elb.amazonaws.com  #the blogging app url link to probe
    relabel_configs:
      - source_labels: [__address__]
        target_label: __param_target
      - source_labels: [__param_target]
        target_label: instance
```

```
    - target_label: __address__
      replacement: 18.175.135.0:9115  # Blackbox Exporter address
```

Make sure to change the blackbox exporter address ***and*** blog app url link***to that which points to your VM IP. and change the blog app url link***

Run Prometheus > prometheus --config.file=prometheus.yaml

c.> Set Up Grafana: Grafana will be used for visualizing metrics collected by Prometheus.

```
# Update package list and install necessary dependencies
sudo apt-get update
sudo apt-get install -y software-properties-common curl

# Add Grafana GPG key
curl https://packages.grafana.com/gpg.key | sudo apt-key add -

# Add Grafana APT repository
sudo add-apt-repository "deb https://packages.grafana.com/oss/deb stable main"

# Update package list again and install Grafana
sudo apt-get update
sudo apt-get install -y grafana

# Start Grafana service
sudo systemctl start grafana-server

# Enable Grafana to start on boot
sudo systemctl enable grafana-server

# Print the status of Grafana service
sudo systemctl status grafana-server
```

d.> Verify Installations: Do this on the termline

```
#for blackbox
curl http://localhost:9115
```

#for blackbox exporter metrics

curl http://localhost:9115/metrics

#for prometheus

curl http://localhost:9090/metrics

#for grafana

sudo systemctl status grafana-server

Open grafana and prometheus using your VM IP and their port

#for grafana

http://<monitoring-vm-ip>:3000

#for prometheus

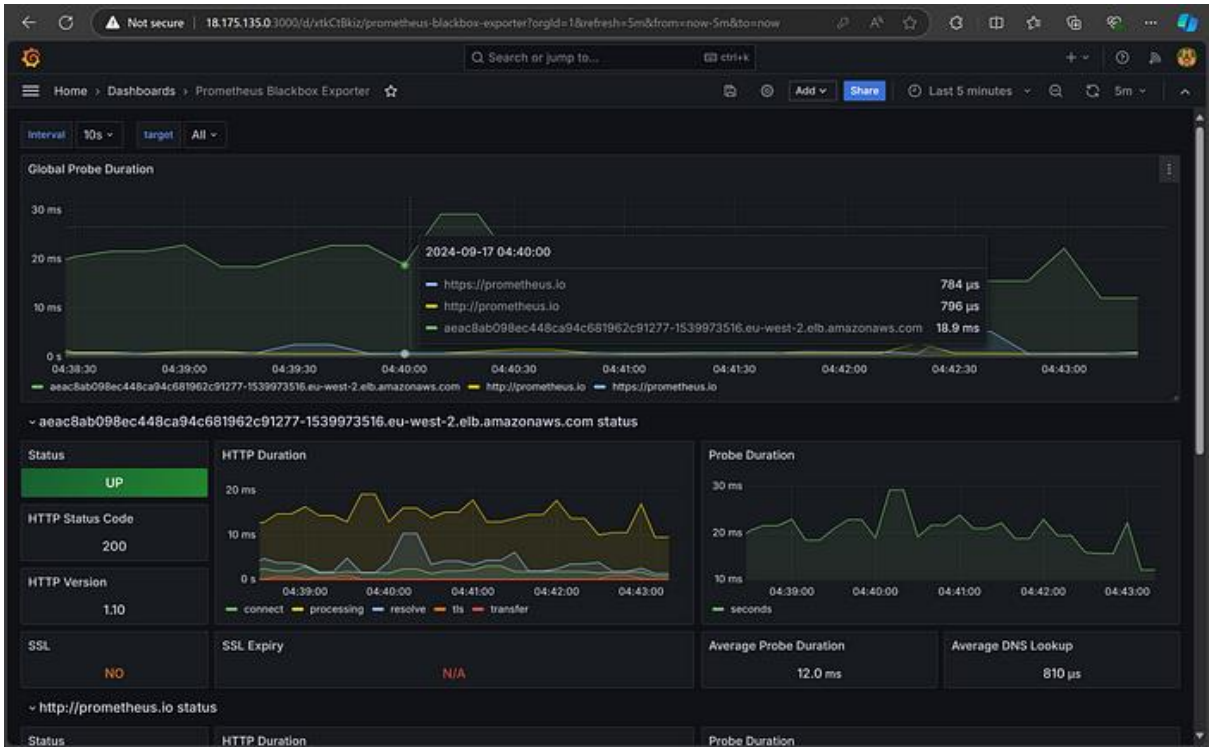
http://<monitoring-vm-ip>:9090

#for blackbox

http://<monitoring-vm-ip>:9115

e.> Add data source and create dashboard:

- Open Grafana in your web browser: > Log in with the default credentials (`admin/admin`) > click on Administration > Add Prometheus as a data source > Set the URL to your Prometheus instance > Click Save & Test.
- On the left pane click on 'import dashboard' > for ID type in 7587 and click Load > select a data source (prometheus) > click import



- Prometheus: collects and stores metrics.
- Blackbox Exporter: probes your application endpoints and provides metrics to Prometheus.
- Grafana: visualizes the metrics collected by Prometheus.